Exercício 1

Considere a declaração:

```
.data
vetor: .word 12, 25, 8, 17, 30, 14, 9, 21
```

Escreva um programa em Assembly MIPS que percorra o vetor e calcule a soma de todos os seus elementos.

Requisitos:

- Utilizar um laço de repetição.
- Utilizar registradores para o índice e o acumulador.
- Considerar que cada elemento ocupa 4 bytes.
- Imprimir a soma calculada.
- Resultado esperado: 136.

Exercício 2

Considere o vetor:

```
.data
vetor: .word 10, -5, 8, -12, 0, -3, 15, -1, 7, -9
```

Implemente um programa que conte quantos elementos negativos existem no vetor.

Requisitos:

- Percorrer todos os elementos.
- Utilizar comparações condicionais.
- Incrementar um contador somente quando o elemento for negativo.
- Imprimir a quantidade encontrada.

Exercício 3

Considere o vetor:

```
.data
vetor: .word 10, 20, 30, 40, 50, 60
```

Escreva um programa que inverta a ordem dos elementos do vetor na própria memória, sem declarar um segundo vetor.

O programa deverá produzir:

60 50 40 30 20 10

Exercício 4

Considere o vetor:

```
.data
vetor: .word 15, 8, 42, 19, 7, 42, 31, 12
```

Solicite ao usuário um número inteiro e procure sua ocorrência no vetor.

O programa deverá imprimir o índice da primeira ocorrência ou -1, caso o elemento não seja encontrado.

Exemplo: se o usuário informar 42, o programa deverá imprimir 2, considerando índices iniciados em zero.

Exercício 5

Considere o vetor:

```
.data
vetor: .word 35, 12, 48, 7, 23, 19
```

Implemente o algoritmo de ordenação por seleção (Selection Sort) em ordem crescente.

Requisitos:

- Encontrar o menor elemento da parte ainda não ordenada.
- Trocar esse elemento com o primeiro elemento dessa parte.
- Repetir o procedimento até ordenar o vetor.
- Imprimir o vetor ordenado.
- Resultado esperado: 7 12 19 23 35 48.

Exercício 6

Escreva um programa que leia um número inteiro positivo e verifique se ele é uma potência de 2.

O programa deverá imprimir `1` se o número for uma potência de 2 e `0` caso contrário. Considere como entrada válida um inteiro positivo representável em 32 bits. Exemplo:

Entrada	Saída
8	1
16	1
12	0
1	1
0	0

Exercício 7

Considere a string:

```
.data
texto: .asciiz "Arquitetura de Computadores"
```

Escreva um programa que conte quantas vogais existem na string, desconsiderando diferenças entre letras maiúsculas e minúsculas.

Requisitos:

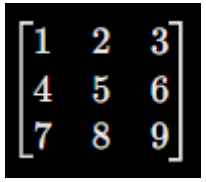
- Percorrer a string caractere por caractere.
- Utilizar lbu para carregar os caracteres.
- Encerrar a leitura ao encontrar \0.
- Contabilizar as vogais a, e, i, o e u, independentemente de maiúsculas ou minúsculas.
- Imprimir a quantidade encontrada.

Exercício 8

Considere a matriz quadrada:

```
.data
matriz: .word 1, 2, 3, 4, 5, 6, 7, 8, 9
```

Considere que ela representa a matriz:



Escreva um programa que calcule e imprima:

- A soma da diagonal principal.
- A soma da diagonal secundária.
- Resultados esperados: diagonal principal = 15; diagonal secundária = 15.

Exercício 9

Considere o vetor:

```
.data
vetor: .byte 10, 20, 10, 30, 10, 40, 20, 50, 10, 60
```

Solicite ao usuário um valor entre 0 e 255 e conte quantas vezes esse valor aparece no vetor.

Requisitos:

- Utilizar lbu para carregar os elementos.
- Comparar cada elemento com o valor informado.
- Contar todas as ocorrências.
- Imprimir o resultado.
- Exemplo: para a entrada 10, a saída será 4.

Exercício 10

Escreva um programa que leia cinco números inteiros e determine o maior deles utilizando um procedimento chamado maximo.

O procedimento deverá receber os endereços ou os valores necessários por meio dos registradores de argumentos, conforme a estratégia escolhida, e retornar o maior valor em $v0.

Requisitos:

- Utilizar jal para chamar o procedimento.
- Utilizar jr $ra para retornar.
- Não calcular o máximo diretamente no programa principal.
- Imprimir o valor retornado.

Atividade Avaliativa

1. Gerar 1 arquivo “.asm” para cada exercício proposto.
2. Compactar os arquivos em “.zip”.
3. Enviar o “.zip” no link correspondente no NEAD.

OBS: A atividade pode ser feita em grupos com até 3 pessoas.